JC4002 Knowledge Representation

Day 5: Prolog II

Dr Ferdian Jovan ferdian.jovan@abdn.ac.uk
Dr Aiden Durrant aiden.durrant@abdn.ac.uk
Dr Jiangtian Nie jiangtian.nie@abdn.ac.uk

Day 5
Prolog II

1. **Matching vs Unification**
2. Structured Data

UNIVERSITY OF ABERDEEN

Matching "="

- Matching is one of Prolog's built-in predicates.
- Although *matching* uses an equivalent symbol (=), it does not mean "is equivalent to".
- Its role is to match *Variables* with *Atoms* in KB
- Example:
    - | ?- car = lorry.

Matching "="

- Matching is one of Prolog's built-in predicates.
- Although *matching* uses an equivalent symbol (=), it does not mean "is equivalent to".
- Its role is to match *Variables* with *Atoms* in KB
- Example:
  - | ?- car = lorry.
  - no
  - | ?- lorry = lorries.

Matching "="

- Matching is one of Prolog's built-in predicates.
- Although *matching* uses an equivalent symbol (=), it does not mean "is equivalent to".
- Its role is to match *Variables* with *Atoms* in KB
- Example:
  - | ?- car = lorry.
  - no
  - | ?- lorry = lorries.
  - no
  - | ?- Lorry = truck.

Matching "="

- Matching is one of Prolog's built-in predicates.
- Although *matching* uses an equivalent symbol (=), it does not mean "is equivalent to".
- Its role is to match *Variables* with *Atoms* in KB
- Example:
  - | ?- car = lorry.
  - no
  - | ?- lorry = lorries.

```
l     o     r     r     y
=     =     =     =     !=
l     o     r     r     i     e     s
```

  - no
  - | ?- Lorry = truck.
  - Lorry = truck ? Why? Lorry is a variable.
- Prolog treats all these queries just as it did the one before them and compares the two words, letter by letter.

Matching "="

- Matching in Prolog is similar to assigning variables in other procedural languages (e.g. Python)
- Example:
  - | ?- Lorry = truck.
    - Lorry = truck ?
  - | ?- truck = Lorry.

Matching "="

- Matching in Prolog is similar to assigning variables in other procedural languages (e.g. Python)
- Example:
  - | ?- Lorry = truck.
    - Lorry = truck ?
  - | ?- truck = Lorry.
    - Lorry = truck ? (Variable doesn't have to be on the left side of "=")
  - | ?- Lorry = Truck, Lorry = volvo

Matching "="

- Matching in Prolog is similar to assigning variables in other procedural languages (e.g. Python)
- Example:
  - | ?- Lorry = truck.
    - Lorry = truck ?
  - | ?- truck = Lorry.
    - Lorry = truck ? (Variable doesn't have to be on the left side of "=")
  - | ?- Lorry = Truck, Lorry = volvo
    - Lorry = volvo, Truck = volvo ? (Both Lorry & Truck are assigned to volvo)
- A variable in Prolog can only have one value during the execution of a query (unlike in many other languages).
  - | ?- Car = nova, fiesta = Car.

Matching "="

- Matching in Prolog is similar to assigning variables in other procedural languages (e.g. Python)
- Example:
  - | ?- Lorry = truck.
    - Lorry = truck ?
  - | ?- truck = Lorry.
    - Lorry = truck ? (Variable doesn't have to be on the left side of "=")
  - | ?- Lorry = Truck, Lorry = volvo
    - Lorry = volvo, Truck = volvo ? (Both Lorry & Truck are assigned to volvo)
- A variable in Prolog can only have one value during the execution of a query (unlike in many other languages).
  - | ?- Car = nova, fiesta = Car.
    - no

Instantiation, Matching, and Unification

- The behaviour of "=" has two facets, *instantiation* and *matching*.
- Instantiation is the way in which variables acquire a value.
  - Once instantiated, can't be reinstantiated
- Matching is used to compare two terms.
- Both these processes are internally used by Prolog to do *unification.*
  - Unification requires a term on either side of it.
  - (Variable = atom) or (atom = Variable) is called instantiation.
  - (atom = atom) or (Variable1 = Variable2) is called matching.
- If both terms are instantiated variables, then the values of the variables are matched just as if they were atoms.
  - | ?- Car1 = nova, Car2 = nova, Car1 = Car2.
    - Car1 = nova, Car2 = nova ?
  - | ?- Car1 = nova, fiesta = Car2, Car1 = Car2.

Instantiation, Matching, and Unification

- The behaviour of "=" has two facets, *instantiation* and *matching*.
- Instantiation is the way in which variables acquire a value.
  - Once instantiated, can't be reinstantiated
- Matching is used to compare two terms.
- Both these processes are internally used by Prolog to do *unification.*
  - Unification requires a term on either side of it.
  - (Variable = atom) or (atom = Variable) is called instantiation.
  - (atom = atom) or (Variable1 = Variable2) is called matching.
- If both terms are instantiated variables, then the values of the variables are matched just as if they were atoms.
  - | ?- Car1 = nova, Car2 = nova, Car1 = Car2.
    - Car1 = nova, Car2 = nova ?
  - | ?- Car1 = nova, fiesta = Car2, Car1 = Car2.
    - no

Matching without unification

- What if we require matching without unification. Use “==“.
- This matches two terms, but to succeed, the two terms have to already have identical values.
- Example:
  - | ?- munich == munich.
    - yes
  - | ?- italy == Country.

Matching without unification

- What if we require matching without unification. Use "==".
- This matches two terms, but to succeed, the two terms have to already have identical values.
- Example:
  - | ?- munich == munich.
    - yes
  - | ?- italy == Country.
    - no (atom vs uninstantiated variable)
  - | ?- Country = italy, Country == italy.

Matching without unification

- What if we require matching without unification. Use "==".
- This matches two terms, but to succeed, the two terms have to already have identical values.
- Example:
  - | ?- munich == munich.
    - yes
  - | ?- italy == Country.
    - no (atom vs uninstantiated variable)
  - | ?- Country = italy, Country == italy.
    - Country = italy ?
  - | ?- Variable1 == Variable2.

Matching without unification

- What if we require matching without unification. Use "==".
- This matches two terms, but to succeed, the two terms have to already have identical values.
- Example:
  - | ?- munich == munich.
    - yes
  - | ?- italy == Country.
    - no (atom vs uninstantiated variable)
  - | ?- Country = italy, Country == italy.
    - Country = italy ?
  - | ?- Variable1 == Variable2.
    - no (check if these variables point to the same memory)
  - | ?- Variable1 = Variable2, Variable1 == Variable2.

Matching without unification

- What if we require matching without unification. Use "==".
- This matches two terms, but to succeed, the two terms have to already have identical values.
- Example:
  - | ?- munich == munich.
    - yes
  - | ?- italy == Country.
    - no (atom vs uninstantiated variable)
  - | ?- Country = italy, Country == italy.
    - Country = italy ?
  - | ?- Variable1 == Variable2.
    - no (check if these variables point to the same memory)
  - | ?- Variable1 = Variable2, Variable1 == Variable2.
    - Variable1 = Variable2 ? (variables have been unified, hence pointing the same memory)

Day 5
Prolog II

1. Matching vs Unification
2. **Structured Data**

UNIVERSITY OF ABERDEEN

Structured Objects

- In Prolog, we used compound terms (i.e. functors / atoms with > 0 arity) to represent *relationships* between simple data objects.
- Matching and unification are done to the arguments of compound terms.
- Example: (assume we have a fact *mammal(gorilla, jambo)* in our KB)
  - | ?- mammal(Name, Species).
    - Name = gorilla, Species = jambo ?
  - | ?- mammal(jambo, gorilla).

Structured Objects

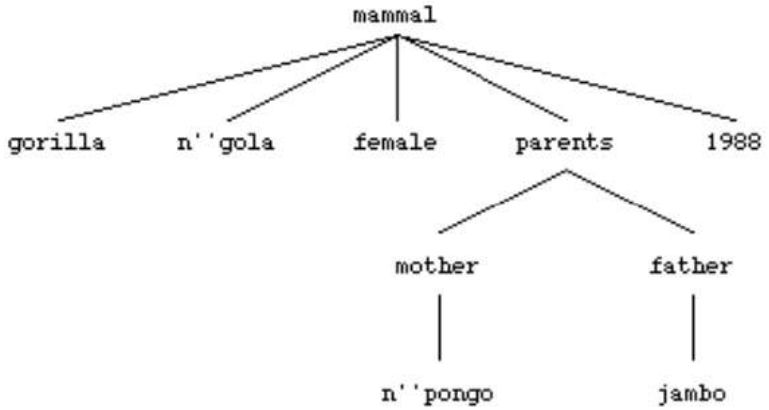
- In Prolog, we used compound terms (i.e. functors / atoms with > 0 arity) to represent *relationships* between simple data objects.
- Matching and unification are done to the arguments of compound terms.
- Example: (assume we have a fact *mammal(gorilla, jambo)* in our KB)
  - | ?- mammal(Name, Species).
    - Name = gorilla, Species = jambo ?
  - | ?- mammal(jambo, gorilla).
    - no
- When it matches arguments, Prolog matches them firstly on their position and then it matches atomic objects.
- Can we have compound terms within compound terms? Yes, using a structure.

Structured Objects

- Can we have compound terms within compound terms? Yes, using a structure.
- How to add ‘Parents’ information to the mammal structure?
- One option is to create a new structured object called ‘parents’ and add it as part of the argument for ‘mammal’ predicate.
  - parents('npongo', jambo).
  - mammal(gorilla, 'ngola', female, parents('npongo', jambo), 1988).
- Which one is the father? Which one is the mother?
  - father(jambo).
  - mother('npongo').
- Two final options:
  - mammal(gorilla, 'ngola', female, mother('npongo'), father(jambo), 1988).
  - mammal(gorilla, 'ngola', female, parents(mother('npongo'), father(jambo)), 1988).

Querying Structured Objects

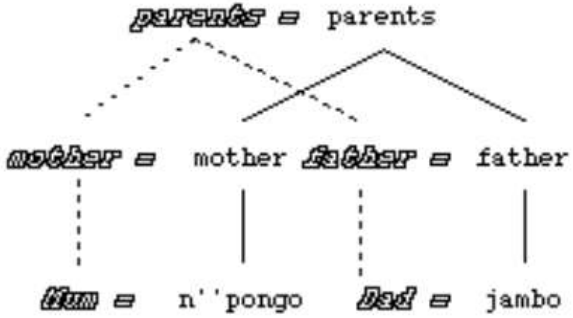
- Structured objects can be drawn as trees.
- Non-terminal nodes are functors; terminal nodes / leaves are atomics or variables.
- mammal(gorilla, 'ngola', female, parents(mother('npongo'), father(jambo)), 1988).



- To query with structured objects, we have to ensure that each item in the nested structured object is covered by either a term or a variable.

Querying Structured Objects

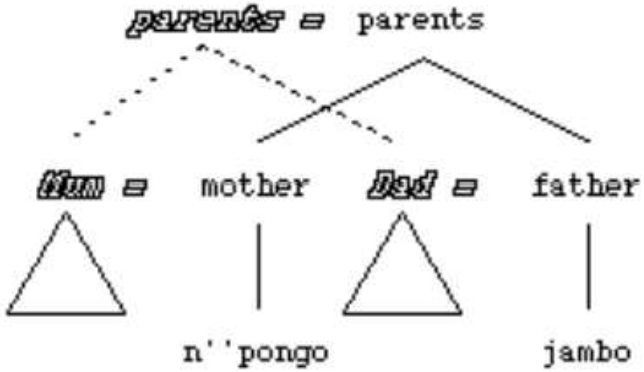
- Example KB: parents(mother('npongo'), father(jambo)).
- Query 1: | ?- parents(mother(Mum), father(Dad))



- Query is in outline italic text connected by dashed lines.
- Each node in the original tree has a corresponding node in the tree that diagrams the query.

Querying Structured Objects

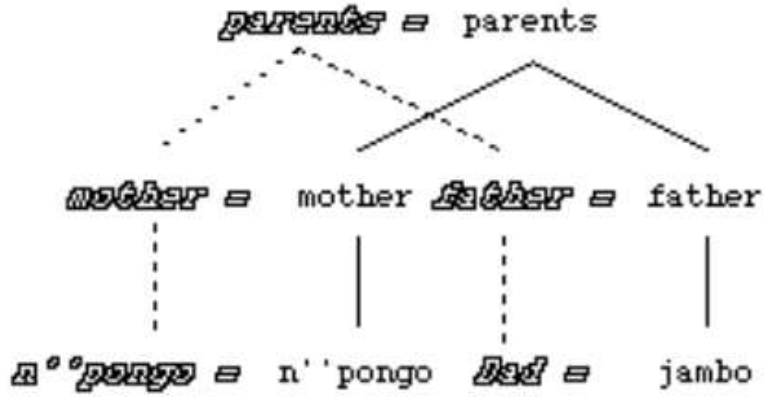
- Example KB: parents(mother('npongo'), father(jambo)).
- Query 2: | ?- parents(Mum, Dad)



- Variables are used to stand for non-terminal nodes.
- In Prolog's response to the query is shown by a variable being unified with a structure, e.g.: Mum = mother('npongo').
- Empty triangle was used to indicate that the variable will stand for the node and everything growing from it.

Querying Structured Objects

- Example KB: parents(mother('npongo'), father(jambo)).
- Query 3: | ?- parents(mother('npongo'), father(Dad))



- Similar to Query 1, but differs only in one of the terminal node.
- One terminal node is represented by an atom ('npongo').

Querying > 1 Goal Through Variables

- Suppose our database includes information not only about animals but also the location and opening times of zoos:
  - zoo(**jersey**, address('Les Augres Manor, Trinity, Jersey'), open(10), close(dusk)).
- Assume that, we have only the following animal recorded in the database:
  - mammal(gorilla, 'ngola', female, parents(mother('npongo'), father(jambo)), 1988, **jersey).**
- To link these two facts, the common argument is the name of the zoo.
- We can use a single variable to query and link these two together.
  - | ?- mammal(gorilla, Name, Sex, Parents, DoB, **Zoo**), zoo(**Zoo**, Address, Open, Close).
  - What will the output look like? Try it using https://swish.swi-prolog.org/
- Goals are joined by the use of the comma "," (often called the *conjunction*)

Querying > 1 Goal Through Variables

- We can use a single variable to query and link these two together.
  - | ?- mammal(gorilla, Name, Sex, Parents, DoB, **Zoo**), zoo(**Zoo**, Address, Open, Close).
  - What will the output look like? Try it using https://swish.swi-prolog.org/
- Goals are joined by the use of the comma "," (often called the *conjunction*)
- We can use "_" to represent "anonymous variable".
- "_" is used wherever we aren't interested in the value that a variable is instantiated to.
  - | ?- mammal(gorilla, _, _, _, _, **Zoo**), zoo(**Zoo**, _, Open, _).
  - Zoo = jersey, Open = open(10) ?
- Do you realise that "_" can have several values at the same time? Why is that?

Summary

- Understanding matching, instantiation, and unification
- Structured objects within structured objects
- Querying structured objects

For today practical:

- Install SWI-Prolog to your computer.